

Hoja de Evaluación: Maze Generator

Transcripción estructurada del checklist oficial (42 Málaga) y Plan Estratégico de Defensa

¡REGLA DE ORO DE 42! Criterios de "Fallo Inmediato" (Grade: 0)

La evaluación se detendrá con una calificación de **0** si falta algún archivo obligatorio, si no se cumple el formato exacto de la primera línea del README, si las herramientas de linter (flake8/mypy) arrojan errores, o si el programa sufre un *crash* inesperado durante las pruebas de gestión de errores.

1. Requisitos Iniciales y Normas de Estilo

El primer paso del corrector consiste en verificar la existencia de los componentes mínimos en el repositorio Git y ejecutar los análisis estáticos de código:

Elemento / Herramienta	Requisito del Subject
Ficheros Obligatorios	Debe contener exactamente: <code>README.md</code> , <code>a_maze_ing.py</code> , el paquete reutilizable <code>mazegen-*.tar.gz</code> y/o <code>.whl</code> , un archivo de configuración predeterminado y todos los componentes para reconstruir el paquete.
Norma de Estilo (flake8)	Todos los archivos de Python del proyecto deben pasar el linter <code>flake8</code> sin una sola advertencia o error.
Tipado Estricto (mypy)	El análisis estático con <code>mypy</code> no debe devolver errores. Se exige tipado de datos explícito en las funciones y módulos desarrollados.

2. Estructura Obligatoria del README.md

El archivo `README.md` actúa como contrato técnico. La ausencia de cualquiera de los siguientes puntos anula el proyecto (Grade 0):

- **Primera línea estricta:** Debe estar en cursiva y con la sintaxis exacta:

```
*This project has been created as part of the 42 curriculum by <login>, <login>.*
```
- **Description:** Sección con el propósito general del proyecto y una breve descripción general.
- **Instructions:** Detalles precisos sobre cómo compilar, instalar y/or ejecutar la herramienta y el paquete.
- **Resources & AI:** Listado de referencias externas (documentación, tutoriales). **Obligatorio:** Explicar explícitamente cómo se utilizó la Inteligencia Artificial, detallando en qué tareas y en qué partes del proyecto se aplicó.
- **Configuration file:** Descripción completa de las variables del archivo de configuración.
- **Algorithm:** Algoritmo de laberintos elegido y la justificación razonada de su elección.
- **Module Documentation:** Documentación breve y concisa sobre el funcionamiento del módulo reutilizable de generación de laberintos.
- **Team and Project Management:**
 - Roles desempeñados por cada miembro del equipo.

- Planificación inicial estimada frente a la evolución real hasta el final del proyecto.
- Qué funcionó bien y qué aspectos podrían mejorarse.
- ¿Se usaron herramientas específicas? Indicar cuáles (ej. Trello, Jira, Notion, etc.).

3. Uso Estándar, Interfaz e Interactividad

El script principal `a_maze_ing.py` debe ejecutarse correctamente usando el archivo de configuración por defecto.

Visualización

Al arrancar, el programa debe mostrar el laberinto generado aleatoriamente. Se permite tanto una representación visual estructurada en la propia terminal como el despliegue de una ventana gráfica dedicada (usando librerías como tkinter, pygame, etc.). Debe existir un mecanismo claro de interacción con el usuario.

Menú Interactivo (Controles Mandatorios)

El evaluador controlará minuciosamente que se puedan realizar las siguientes interacciones:

1. **Regenerar:** Un botón o tecla que genere y pinte instantáneamente un nuevo laberinto aleatorio.
2. **Camino más corto (Toggle):** Un control para mostrar u ocultar visualmente la ruta de resolución óptima desde la entrada hasta la salida.
3. **Cambio de color:** Un control interactivo para modificar el color de los muros del laberinto. (Se aprecian extras como cambiar el color del patrón "42" o la opción directa de salir).

4. Archivo de Configuración y Fichero de Salida

Formato del Archivo de Configuración

- Los comentarios deben comenzar estrictamente con el carácter `#`.
- Cada línea con datos debe seguir el formato estructurado `KEY=VALUE` (se acepta el uso de minúsculas).
- Las claves obligatorias que deben figurar son: `WIDTH`, `HEIGHT`, `ENTRY`, `EXIT`, `OUTPUT_FILE` y `PERFECT`.

Prueba de Estrés: Gestión de Errores (¡Punto Crítico!)

El corrector editará manualmente tu configuración para forzar fallos de la siguiente manera: eliminando claves obligatorias, alterando el formato quitando el signo '=', escribiendo letras donde debe ir un número, usando booleanos falsos o corrompiendo la tupla de enteros de ENTRY/EXIT. **Tu código debe capturar estas excepciones limpiamente**, dar un mensaje explicativo y finalizar de forma controlada. Si salta un Traceback descontrolado de Python, estás suspendido.

Estructura del Archivo de Salida (Output File)

Cuando se ejecuta el programa, el primer laberinto generado y mostrado en pantalla debe guardarse de forma idéntica en la ruta especificada en `OUTPUT_FILE`, cumpliendo este patrón estricto:

1. `HEIGHT` líneas de texto, compuestas por `WIDTH` caracteres cada una (representando los muros y pasillos).
2. Una línea completamente vacía.
3. La tupla que define la entrada `ENTRY`.
4. La tupla que define la salida `EXIT`.
5. La secuencia exacta del camino más corto descrita mediante caracteres de dirección cardinal: `N` (Norte), `E` (Este), `S` (Sur), `W` (Oeste).

Nota: El evaluador pasará un script de validación oficial provisto por el subject para verificar que las paredes tienen coherencia matemática y que la ruta indicada en el archivo coincide exactamente con la solución visual.

5. Requisitos del Módulo Generador y Empaquetado

Reglas de Construcción del Laberinto

- **Aleatoriedad y Replicabilidad:** Uso correcto de la generación pseudoaleatoria (módulo `random`). Si se provee una misma semilla (seed) en el archivo o código, el laberinto generado debe ser exactamente el mismo.
- **Robustez de Parámetros:** El sistema debe comportarse de forma lógica y segura si los parámetros son inconsistentes (ej. dimensiones negativas o coordenadas de entrada/salida situadas fuera del mapa).
- **Conectividad y Muros:** Todo el laberinto debe estar rodeado por muros perimetrales cerrados. Todas las celdas válidas deben ser alcanzables, a excepción de las celdas ocupadas por el patrón "42".
- **Restricción de Espacios Abiertos:** Está prohibido que existan áreas abiertas de 3x3 celdas o más que carezcan de muros interiores. El corrector te preguntará cómo implementaste esta restricción en tu algoritmo.
- **El Patrón "42":** El diseño del laberinto debe integrar el número de la escuela ("42") en sus pasillos/muros internos. Si las dimensiones configuradas son demasiado pequeñas para albergarlo, el programa debe mostrar un aviso informativo claro en la terminal.
- **Laberinto Perfecto:** Si la propiedad `PERFECT` está en `True`, el laberinto resultante debe ser matemáticamente perfecto: sin bucles o caminos cerrados alternativos, garantizando que existe única y exclusivamente un único camino entre dos celdas cualesquiera.

Prueba de Modularidad Reutilizable

El evaluador comprobará que tu librería está bien empaquetada siguiendo estos pasos:

- Creará un entorno virtual limpio (`virtualenv` o `venv`).
- Ejecutará los comandos necesarios para compilar y empaquetar tu código, recreando el archivo `mazegen-*.tar.gz` o `.whl`.
- Creará un **segundo entorno virtual diferente**, instalará en él tu paquete generado mediante `pip install`, y comprobará que el script ejecutable de la raíz (`a_maze_ing.py`) funciona perfectamente llamando a la biblioteca instalada.

6. Bonificaciones y Notas de Cierre

Los puntos extra (escala de 0 a 5) solo se evaluarán si toda la sección mandatoria anterior ha obtenido una puntuación perfecta. Se pueden presentar hasta 5 bonus sustanciales. El subject propone ideas como soportar múltiples algoritmos de generación intercambiables o añadir animaciones paso a paso del proceso de generación.

Plan Estratégico y Recomendaciones Técnicas

Consejos orientados a superar la evaluación en 42 Málaga con la máxima calificación

Recomendación 1: Blindaje Total contra el Grade 0 (Linter y Tipado)

No confíes en comprobar el código manualmente antes de la evaluación. Configura un script automatizado o un hook de Git que valide tu código de forma estricta. Asegúrate de ejecutar localmente:

```
$ flake8 . --max-line-length=88
$ mypy . --strict --disallow-untyped-defs
```

Evita por completo el uso de `Any` en las anotaciones de `mypy`. Si utilizas librerías externas de gráficos que carezcan de stubs de tipado, añade los comentarios correspondientes `# type: ignore` con moderación y justificación técnica.

Recomendación 2: Arquitectura del Parser de Configuración (Anti-Crash)

Para evitar que el programa falle estrepitosamente ante configuraciones maliciosas, no parsees las líneas de texto de forma rudimentaria. Diseña una clase o módulo independiente encargado exclusivamente de validar la configuración. Implementa un bloque estructurado de captura de excepciones generales:

```
try:
    # Lógica de lectura, split('='), casting a int de tuplas, etc.
except (ValueError, KeyError, IndexError) as err:
    print(f"Error crítico en archivo de configuración: {err}", file=sys.stderr)
    sys.exit(1)
```

Un retorno limpio con un código de salida `1` (o cualquier número distinto de cero) demuestra profesionalidad y salvará tu evaluación, transformando un potencial suspenso inmediato en una sección superada con éxito.

Recomendación 3: Algoritmia para Laberintos Perfectos y Control de Espacios 3x3

Para cumplir la condición matemática de un laberinto perfecto cuando `PERFECT=True`, implementa el algoritmo de **Estructuración por Árbol de Expansión Mínima (Kruskal o Prim)** o un **DFS con Backtracking** basado en una pila. Al garantizar que cada celda se conecta mediante un único camino, evitarás la formación de ciclos y habitaciones vacías de forma nativa.

Para los casos imperfectos, puedes generar un laberinto perfecto y posteriormente eliminar de manera controlada un porcentaje de muros aleatorios. Inmediatamente después de este proceso, pasa una función de escaneo que analice submatrices de 3x3 celdas; si detecta una zona completamente diáfana, inserta un muro estratégico de manera automática para cumplir la restricción del subject.

Recomendación 4: Integración Limpia del Patrón "42"

Una estrategia muy sólida consiste en definir el patrón "42" de forma estática en una matriz binaria o máscara (por ejemplo, una cuadrícula fija de 5x7 celdas). Al inicializar tu mapa, puedes imprimir esta máscara geométrica en el centro geométrico del laberinto, marcando esas celdas como muros o pasillos inalterables, y posteriormente forzar al algoritmo de generación a construir el resto del laberinto a su alrededor, asegurando que las entradas y salidas del patrón queden conectadas al flujo principal.

Recomendación 5: Ensayo General del Flujo de Empaquetado

Dado que el evaluador compilará e instalará tu módulo en dos entornos virtuales aislados, tú debes replicar ese proceso exacto paso a paso en tu máquina antes de subir el proyecto. Utiliza herramientas estándar modernas como `setuptools` (con un archivo `setup.py` o un archivo moderno `pyproject.toml`). Ejecuta la compilación con `python3 -m build` para generar los archivos de distribución en la carpeta `dist/`, y comprueba la portabilidad instalando el archivo wheel resultante mediante un script automatizado de pruebas.